CS182: Introduction to Machine Learning, Fall 2025
Homework 3

(Due on Wednesday, Dec. 10 at 11:59 pm (CST))

Notice:

- Please submit your assignments via Gradescope. The entry code is 428YBY.
- Please make sure you select your answer to the corresponding question when submitting your assignments.
- Each person has a total of five days to be late without penalty for all the assignments. Each late delivery less than one day will be counted as one day.

1. [20 points] [*Multilayer Perceptrons*]You are designing a decoupled controller for a 2-DOF (Degree of Freedom) robotic arm. The network takes the joint angles (θ_1, θ_2) as input and simultaneously outputs the required torques (τ_1, τ_2) for both joints.

   Architecture:

   - Input Layer: 2 neurons ($\mathbf{x} \in \mathbb{R}^2$).
   - Hidden Layer: 2 neurons with *ReLU* activation: $\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$.
   - Output Layer: 2 neurons with *Linear* activation: $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$.

   Parameters: The weights and biases are initialized as follows:

$$\mathbf{W}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}, \quad \mathbf{b}^{(1)} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad \mathbf{b}^{(2)} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

   (a) Given the joint configuration input $\mathbf{x} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$, compute the predicted torque vector $\hat{\mathbf{y}}$. [5 points]

   (b) The optimal torque provided by the physics engine (ground truth) is $\mathbf{y} = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$.

   We define the loss as the Mean Squared Error (scaled):

$$L = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2 = \frac{1}{2}\sum_i (y_i - \hat{y}_i)^2$$

   Using the prediction $\hat{\mathbf{y}}$, compute the gradients of the loss with respect to the first-layer weights $\mathbf{W}^{(1)}$.[10 points]

   (c) Another engineer proposes changing the output activation to a Sigmoid function ($f(z) = \frac{1}{1+e^{-z}}$) to make the output "smoother". Is using a Sigmoid output appropriate for this regression task? Explain why or why not.[5 points]

Solution:

(a) Compute Hidden Layer Pre-activation ($\mathbf{z}^{(1)}$)

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

Substituting the values:

$$\mathbf{z}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$\mathbf{z}^{(1)} = \begin{bmatrix} 1(1) + 1(-2) \\ 1(1) + (-1)(-2) \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 3 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 4 \end{bmatrix}$$

Applying the ReLU activation function $f(z) = \max(0, z)$:

$$\mathbf{h} = \text{ReLU}(\mathbf{z}^{(1)}) = \begin{bmatrix} \max(0, -1) \\ \max(0, 4) \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$$

Using the Linear activation function $\hat{\mathbf{y}} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$:

$$\hat{\mathbf{y}} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 0 \\ 4 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$\hat{\mathbf{y}} = \begin{bmatrix} 2(0) + 1(4) \\ 1(0) + 2(4) \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 4 \\ 8 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$$

Answer: $\hat{\mathbf{y}} = \begin{bmatrix} 5 \\ 8 \end{bmatrix}$

(b) Since the output activation is Linear and the loss is MSE ($L = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$):

$$\delta^{(2)} = \frac{\partial L}{\partial \hat{\mathbf{y}}} = -(\mathbf{y} - \hat{\mathbf{y}}) = \hat{\mathbf{y}} - \mathbf{y}$$

$$\delta^{(2)} = \begin{bmatrix} 5 \\ 8 \end{bmatrix} - \begin{bmatrix} 3 \\ 7 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}$$

The formula is $\delta^{(1)} = (\mathbf{W}^{(2)})\delta^{(2)} \odot f'(\mathbf{z}^{(1)})$, where $\odot$ is the element-wise product. Backpropagate error through weights:

$$(\mathbf{W}^{(2)})\delta^{(2)} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 2(2) + 1(1) \\ 1(2) + 2(1) \end{bmatrix} = \begin{bmatrix} 5 \\ 4 \end{bmatrix}$$

Calculate derivative of ReLU at $\mathbf{z}^{(1)} = \begin{bmatrix} -1 \\ 4 \end{bmatrix}$:

$$f'(\mathbf{z}^{(1)}) = \begin{bmatrix} f'(-1) \\ f'(4) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Combine to find $\delta^{(1)}$:

$$\delta^{(1)} = \begin{bmatrix} 5 \\ 4 \end{bmatrix} \odot \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 4 \end{bmatrix}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \delta^{(1)}\mathbf{x}$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \begin{bmatrix} 0 \\ 4 \end{bmatrix} \begin{bmatrix} 1 & -2 \end{bmatrix} = \begin{bmatrix} 0(1) & 0(-2) \\ 4(1) & 4(-2) \end{bmatrix}$$

Answer:

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \begin{bmatrix} 0 & 0 \\ 4 & -8 \end{bmatrix}$$

(c) **Answer: No.**

(a) **Range Limitation:** The Sigmoid function $f(z) = \frac{1}{1+e^{-z}}$ outputs values strictly in the range $(0, 1)$. In this problem, the ground truth target values are $y = \begin{bmatrix} 3 \\ 7 \end{bmatrix}$. Since 3 and 7 are outside the range $(0, 1)$, a network with a Sigmoid output layer is mathematically incapable of predicting these target values.

(b) **Task Mismatch:** This is a regression task (predicting continuous torque values). Standard practice for regression is to use a **Linear** output layer to allow for unbounded output values. Sigmoid functions are typically used for classification tasks where the output represents a probability.

2. [20 points] [*Support Vector Machines*]

(a) Given the positive class points $x_1 = (1,2)^T$, $x_2 = (2,3)^T$, $x_3 = (3,3)^T$, and the negative class points $x_4 = (2,1)^T$, $x_5 = (3,2)^T$, find the maximum–margin separating hyperplane and the classification decision function. Then, plot the separating hyperplane, the margin boundaries, and the support vectors on a 2D plane. (using sklearn, and provide both the code and the generated results.) [10 points]

(b) A linear support vector machine can also be defined as follows:

$$\min_{w,b,\zeta} \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N} \zeta_i^2$$

subject to

$$y_i(w \cdot x_i + b) \geq 1 - \zeta_i, \quad \forall i = 1, 2, \ldots, N$$

$$\zeta_i \geq 0, \quad \forall i = 1, 2, \ldots, N$$

Find its dual formulation, and please provide detailed derivations. [10 points]

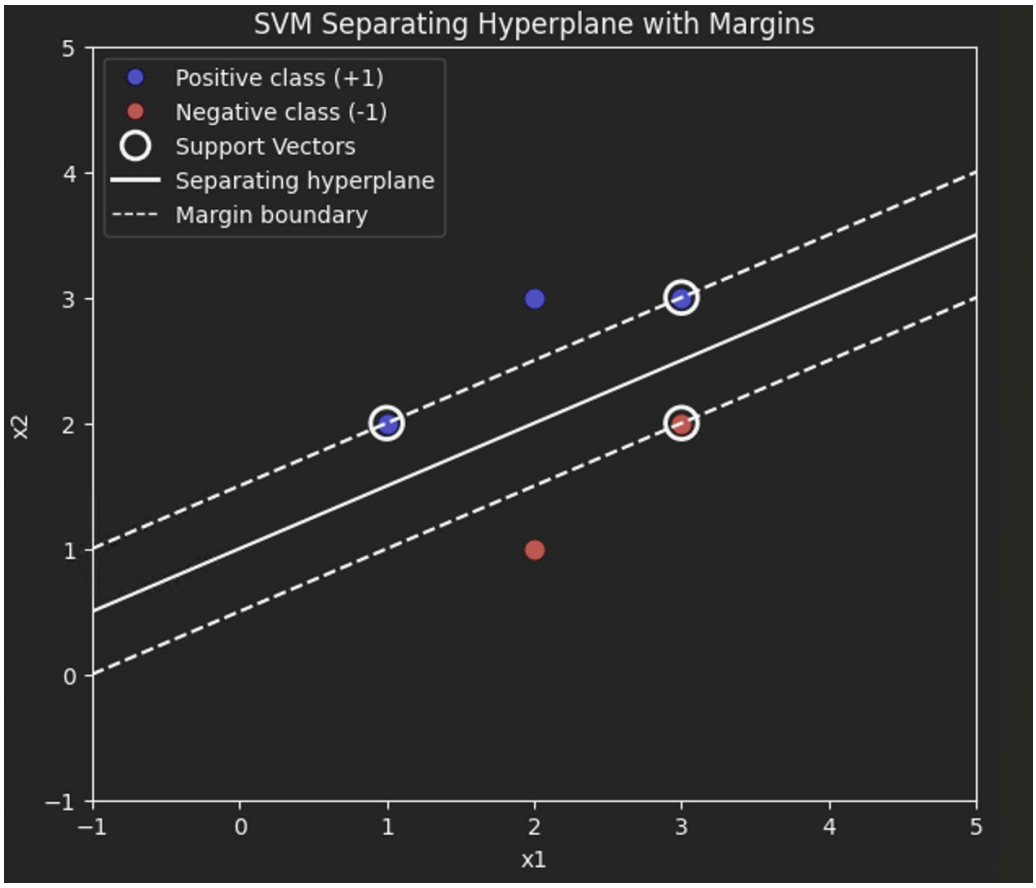
Solution:

(a) Code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from sklearn import svm
# ---------------------------------------------------
# 1. Training data
# ---------------------------------------------------
X = np.array([[1,2],[2,3],[3,3],[2,1],[3,2]])
Y = np.array([1,1,1,-1,-1])
# Train linear SVM
clf = svm.SVC(kernel='linear', C=10000)
clf.fit(X, Y)
# ---------------------------------------------------
# 2. Create meshgrid to evaluate decision function
# ---------------------------------------------------
xx = np.linspace(-1, 5, 200)
yy = np.linspace(-1, 5, 200)
X1, X2 = np.meshgrid(xx, yy)
Z = np.empty(X1.shape)
for (i, j), _ in np.ndenumerate(X1):
    Z[i, j] = clf.decision_function([[X1[i, j], X2[i, j]]])[0]
# Levels: -1 (margin), 0 (hyperplane), +1 (margin)
levels = [-1.0, 0.0, 1.0]
linestyles = ['dashed', 'solid', 'dashed']
colors = 'k'
plt.figure(figsize=(7,6))
# ---------------------------------------------------
# 3. Plot separating hyperplane and margins
# ---------------------------------------------------
plt.contour(X1, X2, Z, levels=levels,
    colors=colors, linestyles=linestyles)
# ---------------------------------------------------
# 4. Plot data points
# ---------------------------------------------------
plt.scatter(X[Y==1][:,0], X[Y==1][:,1],
    c='blue', s=60,
    label='Positive class (+1)')
plt.scatter(X[Y==-1][:,0], X[Y==-1][:,1],
    c='red', s=60,
    label='Negative class (-1)')
# ---------------------------------------------------
# 5. Plot support vectors (white hollow circle)
# ---------------------------------------------------
plt.scatter(clf.support_vectors_[:,0],
    clf.support_vectors_[:,1],
    s=200, facecolors='none', edgecolors='k', linewidths=2.0,
    label='Support vectors')
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('SVM Separating Hyperplane with Margins')
```

```
plt.axis('tight')
# ------------------------------------------------
# 6. Custom Legend (with hollow circle support vectors)
# ------------------------------------------------
legend_elements = [
    Line2D([0],[0], marker='o', color='w',
        markerfacecolor='blue',
        markersize=8, label='Positive class (+1)'),
    Line2D([0],[0], marker='o', color='w',
        markerfacecolor='red',
        markersize=8, label='Negative class (-1)'),
    Line2D([0],[0], marker='o', color='none',
        markerfacecolor='none', markeredgecolor='k',
        markeredgewidth=2, markersize=12,
        label='Support Vectors'),
    Line2D([0],[0], color='k', lw=2,
        label='Separating hyperplane'),
    Line2D([0],[0], color='black', lw=1,
        linestyle='--', label='Margin boundary')
]
plt.legend(handles=legend_elements, loc='upper left')
plt.show()
```

The result is shown in follow figure:



(b) First, rewrite the primal objective into the Lagrangian form:

$$L(w,b,\zeta,\alpha,\mu) = \frac{1}{2}\|w\|^2 + C\sum_{i=1}^{N}\zeta_i^2 - \sum_{i=1}^{N}\alpha_i\left[y_i(w\cdot x_i + b) - 1 + \zeta_i\right] - \sum_{i=1}^{N}\mu_i\zeta_i,$$

subject to

$$\alpha_i \geq 0, \quad \mu_i \geq 0, \quad \forall i = 1,\ldots,N.$$

The Lagrangian form of the primal problem is:

$$\min_{w,b,\zeta}\ \max_{\alpha,\mu}\ L(w,b,\zeta,\alpha,\mu)$$

which can be converted into the dual form:

$$\max_{\alpha,\mu} \min_{w,b,\zeta} L(w, b, \zeta, \alpha, \mu).$$

To compute the dual, first minimize L with respect to w, b, and ζ:

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^{N} \alpha_i y_i x_i = 0,$$

$$\frac{\partial L}{\partial b} = -\sum_{i=1}^{N} \alpha_i y_i = 0,$$

$$\frac{\partial L}{\partial \zeta_i} = 2C\zeta_i - \alpha_i - \mu_i = 0, \quad \forall i.$$

Substituting these conditions back into the Lagrangian yields:

$$\min_{w,b,\zeta} L(w, b, \zeta, \alpha, \mu) = -\frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) + \sum_{i=1}^{N} \alpha_i - \frac{1}{4C^2} \sum_{i=1}^{N} (\alpha_i + \mu_i)^2.$$

Thus, the final dual problem is:

$$\min_{\alpha,\mu} \quad \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) - \sum_{i=1}^{N} \alpha_i + \frac{1}{4C^2} \sum_{i=1}^{N} (\alpha_i + \mu_i)^2$$

subject to

$$\sum_{i=1}^{N} \alpha_i y_i = 0,$$

$$\alpha_i \geq 0, \quad \mu_i \geq 0, \quad \forall i.$$

3. [20 points] [*Kernel Extension*]

In this part, let us try to get a better intuition about kernels themselves.

Definition 1 (Positive definite kernel)**.** *A function $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is called a positive definite kernel iff it is symmetric, that is, $k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}', \mathbf{x})$ for any two objects $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$, and positive definite definite, that is,*

$$\sum_{i=1}^{n} \sum_{j=1}^{n} c_i c_j k(\mathbf{x}_i, \mathbf{x}_j) \geq 0$$

for any $n \geq 0$, any choice of n objects $x_1 \cdots, x_n \in \mathcal{X}$, and any choice of real numbers $c_1, \cdots, c_n \in \mathbb{R}$.

From now on, we only focus on positive definite kernels, and simply call them *kernels*.

Kernels as Inner Product For general objects $\mathbf{x} \in \mathcal{X}$, we first represent each object $\mathbf{x} \in \mathcal{X}$ as a vector $\phi(\mathbf{x}) \in \mathbb{R}^p$, and then defining a kernel for any $\mathbf{x}, \mathbf{x}' \in \mathcal{X}$ by

$$k(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$$

Kernels as Measures of Similarity Kernels are often presented as measures of similarity, in the sense that $k(\mathbf{x}, \mathbf{x}')$ is "large" when $\mathbf{x}$ and $\mathbf{x}'$ are "similar".

(a) Consider the following kernel and give a relevant interpretations as a measure of similarity. You can just describe it in natural language.

$$k(\mathbf{x}, \mathbf{x}') = \exp\{-\frac{d(\mathbf{x}, \mathbf{x}'^2)}{2\sigma^2}\}$$

where σ is a parameter and d is the Euclidean distance.[4 points]

Kernels as Measures of Function Regularity Now we associate kernel with some functional space and norms associated with it. Let k be a kernel on a space $\mathcal{X}$. The set $\mathcal{H}_k$ is defined as the set of function $f : \mathcal{X} \to \mathbb{R}$ of the form

$$f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x})$$

for $n > 0$, a finite number of points $\mathbf{x}_1, \cdots, \mathbf{x}_n \in \mathcal{X}$, and a finite number of weights $\alpha_1, \cdots, \alpha_n \in \mathbb{R}$, together with

$$||f||^2_{\mathcal{H}_k} := \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j k(\mathbf{x}_i, \mathbf{x}_j)$$

The dot product defined for two elements $f(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x})$ and $g(\mathbf{x}) = \sum_{j=1}^{m} \alpha'_j k(\mathbf{x}'_j, \mathbf{x})$ is

$$\langle f, g \rangle = \sum_{i=1}^{n} \sum_{j=1}^{m} \alpha_i \alpha'_j k(\mathbf{x}_i, \mathbf{x}'_j)$$

(b) Express the value $f(\mathbf{x})$ of a function $f \in \mathcal{H}_k$ at a point $\mathbf{x} \in \mathcal{X}$ as a dot product in $\mathcal{H}_k$.[3 points]

(c) Write down the corresponding f, g when $\langle f, g \rangle = k(\mathbf{x}, \mathbf{x}')$.[3 points]

Most kernel methods (including SVMs) have an interpretation in terms of functional analysis and can be defined as algorithms that, given a set of objects $\mathcal{S}$, return a function that solves the equation

$$\min_{f \in \mathcal{H}_k} E(f, \mathcal{S}) + g(\|f\|_{\mathcal{H}_k})$$

(d) (The Representer Theorem) Consider a positive-definite real-valued kernel $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ on a non-empty set $\mathcal{X}$ with a corresponding reproducing kernel space $\mathcal{H}_k$. Let there be given

- a training sample $(x_1, y_1), \ldots, (x_n, y_n) \in \mathcal{X} \times \mathbb{R}$
- a strictly increasing real-valued function $g\colon [0, \infty) \to \mathbb{R}$
- an arbitrary error function $E\colon (\mathcal{X} \times \mathbb{R}^2)^n \to \mathbb{R} \cup \{\infty\}$

which together define the following regularized empirical risk functional on $\mathcal{H}_k$

$$f \mapsto E\left((x_1, y_1, f(x_1)), \ldots, (x_n, y_n, f(x_n))\right) + g\left(\|f\|_{\mathcal{H}_k}\right)$$

Show that any minimizer of the empirical risk

$$f^* = \underset{f \in H_k}{\operatorname{argmin}} \left\{E\left((x_1, y_1, f(x_1)), \ldots, (x_n, y_n, f(x_n))\right) + g\left(\|f\|_{\mathcal{H}_k}\right)\right\}, \quad (*)$$

admits a representation of the form

$$f^*(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i)$$

[10 points]

Solution:

(a) Observe that this kernel is a decreasing function of the Euclidean distance between points, and therefore has a relevant interpretation as a measure of similarity: the larger the kernel $k(\mathbf{x}, \mathbf{x}')$, the closer the points $\mathbf{x}$ and $\mathbf{x}'$ in $\mathcal{X}$.

(b)
$$f(\mathbf{x}) = \langle f, k(\mathbf{x}, \cdot)\rangle$$

(c)
$$f(\cdot) = k(\mathbf{x}, \cdot)$$
$$g(\cdot) = k(\mathbf{x}', \cdot)$$

(When considering the map $\phi : \mathcal{X} \to \mathcal{H}_k$ defined by $\phi(\mathbf{x}) := k(\mathbf{x}, \cdot)$, we get the inner product representation as mentioned before.)

(d) *Proof.* We first decompose any $f \in \mathcal{H}_k$ into a part contained in the span of the kernel functions $k(\mathbf{x}_1), \cdots, k(\mathbf{x}_n)$, and one in the orthogonal complement

$$f(\mathbf{x}) = f_{\|}(\mathbf{x}) + f_{\perp}(\mathbf{x}) = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x}) + f_{\perp}(\mathbf{x})$$

where $\alpha_i \in \mathbb{R}$ and $f_{\perp} \in \mathcal{H}_k$ with $\langle f_{\perp}, k(\mathbf{x}_i, \cdot)\rangle = 0$ for all $i \in [n]$
Note that for any $j \in [n], x$

$$f(\mathbf{x}_j) = \langle f(\cdot), k(\mathbf{x}_j, \cdot)\rangle = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x}_j) + \langle f_{\perp}(\mathbf{x}), k(\mathbf{x}_j, \cdot)\rangle = \sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \mathbf{x}_j)$$

which implies $f_{\perp}$ does not affect the first term of (*), and for all $f_{\perp}$

$$g(\|f\|_{\mathcal{H}_k}) = g\left(\sqrt{\left\|\sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \cdot)\right\|_{\mathcal{H}_k}^2 + \|f_{\perp}\|_{\mathcal{H}_k}^2}\right) \geq g\left(\left\|\sum_{i=1}^{n} \alpha_i k(\mathbf{x}_i, \cdot)\right\|_{\mathcal{H}_k}\right)$$

with equality holds iff $f_{\perp} = 0$ Thus any minimizer of (*) must have $f_{\perp} = 0$, i.e. in the form of

$$f^*(\cdot) = \sum_{i=1}^{n} \alpha_i k(\cdot, x_i)$$

□

4. [20 points] [*Coding*] Complete the 'Problem4.ipynb'. Please include the code you need to write and the output results (which may include values and images) in the PDF file for submission.

Please include the code you need to write and the output results (which may include values and images) in the PDF file for submission.

In this part of homework, you need to implement MLP and SVM using Python in this jupyter notebook

1 [*Multilayer Perceptrons*] Problem (a) [8 points]

[1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from pylab import *
import matplotlib.cm as cm
import warnings
warnings.filterwarnings(action = 'ignore')
from sklearn.model_selection import train_test_split
import sklearn.neural_network as net
```

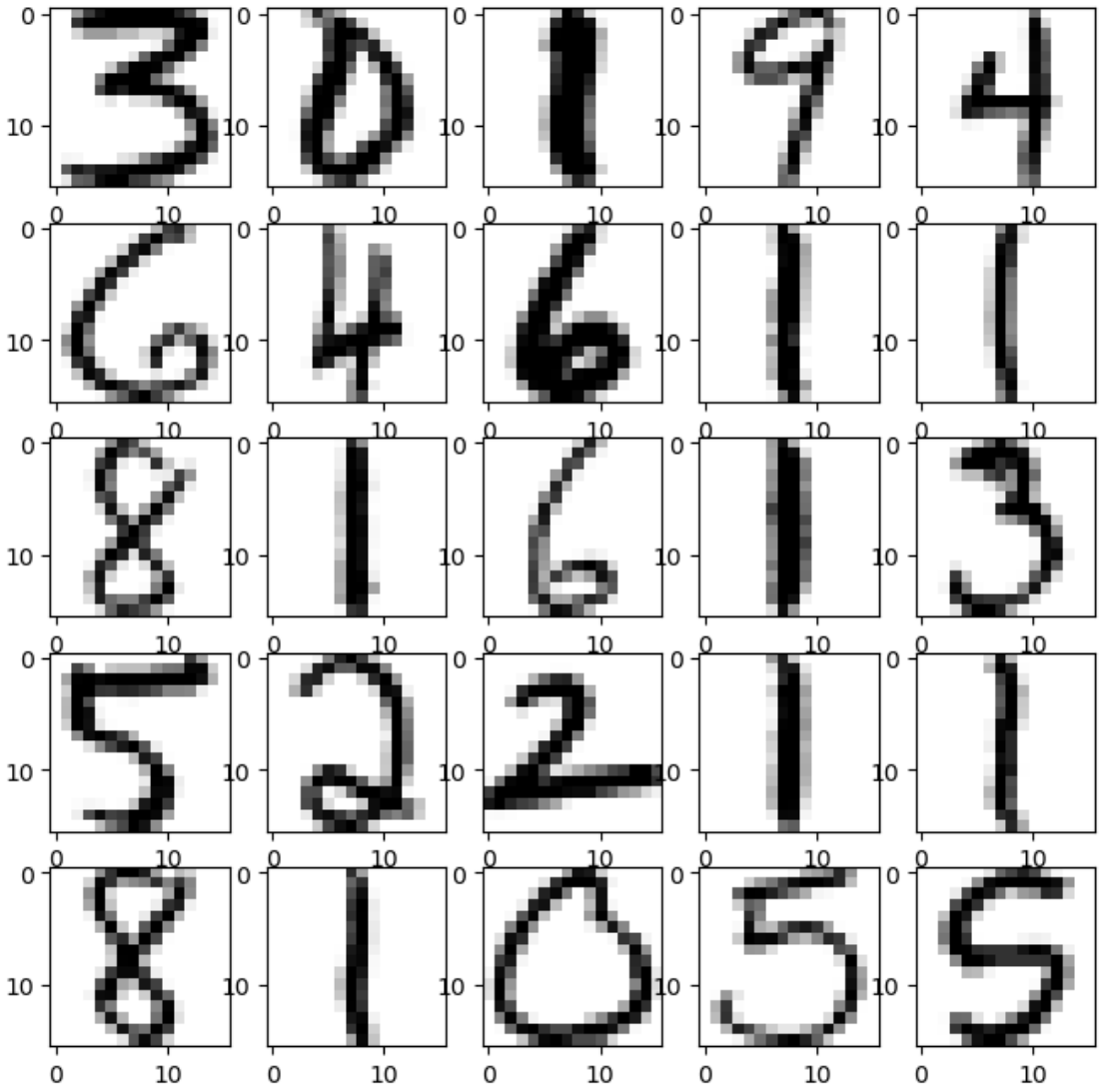
The handwritten Arabic digits are stored in a text file format. The data consists of 16x16 grayscale matrix values, stored from column 2 to column 257. The first column contains the actual digit corresponding to the grayscale matrix data. First, the row data needs to be converted into a 16x16 two-dimensional array.

Use the imshow function from the pylab library to display the matrix data as an image. Image data is usually represented in the third dimension of an array for color.

Visualize data

[ ]:
```
data=pd.read_table('./data/Postal_Code_Data.txt',sep=' ',header=None)
X=data.iloc[:,1:-1]
Y=data.iloc[:,0]
#print(Y.unique())
np.random.seed(1)
ids=np.random.choice(len(Y),25)
plt.figure(figsize=(8,8))
for i,item in enumerate(ids):
    img=np.array(X.iloc[item,]).reshape((16,16))
    plt.subplot(5,5,i+1)
    plt.imshow(img,cmap=cm.gray_r)
```

```
plt.show()
```



[3]:
```
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,train_size=0.60,␣
↪random_state=123)
```

1.1 Use a Three-Layer Neural Network to Recognize Handwritten Arabic Digits [5 points]

- Set the hidden layer nodes to **1, 3, 5, and 19**.
- Set the activation functions to **ReLU** and **Logistic** (sigmoid) functions.
- Compute the **training error** and **testing error** for each case.
- **Use `net.MLPClassifier()` and set the `random_state` parameter to 1.**

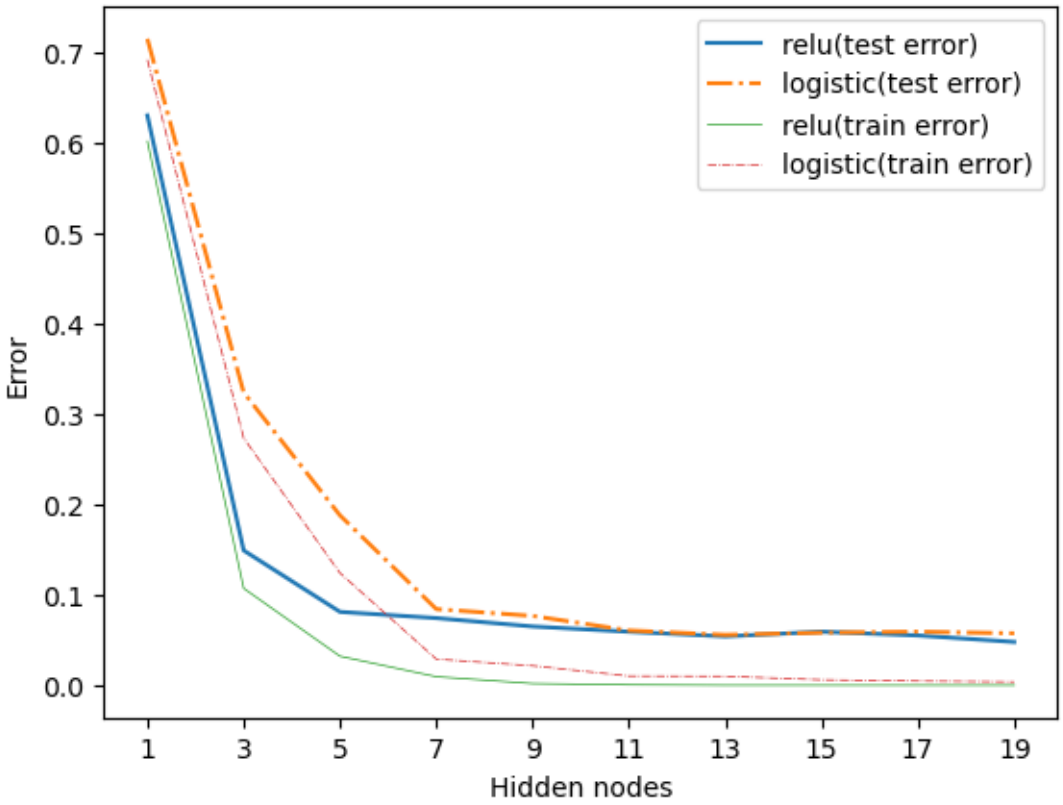
[4]:
```
########## START YOUR CODE HERE #############
nodes=np.arange(1,20,2)
acts=['relu','logistic']
errTrain=np.zeros((len(nodes),2))
errTest=np.zeros((len(nodes),2))
for i,node in enumerate(nodes):
    for j,act in enumerate(acts):
        NeuNet=net.
 ↪MLPClassifier(hidden_layer_sizes=(node,),activation=act,random_state=1)
        NeuNet.fit(X_train,Y_train)
        errTrain[i,j]=1-NeuNet.score(X_train,Y_train)
        errTest[i,j]=1-NeuNet.score(X_test,Y_test)
########## END YOUR CODE HERE ##############
```

1.2 Visualize how the training error and testing error change with the number of hidden nodes under different activation functions. [3 points]

- Use `plt.plot()` .

[6]:
```
########## START YOUR CODE HERE #############
plt.plot(nodes,errTest[:,0],label="relu(test error)",linestyle='-')
plt.plot(nodes,errTest[:,1],label="logistic(test error)",linestyle='-.')
plt.plot(nodes,errTrain[:,0],label="relu(train␣
 ↪error)",linestyle='-',linewidth=0.5)
plt.plot(nodes,errTrain[:,1],label="logistic(train error)",linestyle='-.
 ↪',linewidth=0.5)
plt.xlabel('Hidden nodes')
plt.ylabel('Error')
plt.xticks(nodes)
plt.legend()
########## END YOUR CODE HERE ##############
```

[6]:
```
<matplotlib.legend.Legend at 0x1e748360810>
```



2 [*Support Vector Machines*] Problem (b) [12 points]

[5]:
```
import numpy as np
import pandas as pd
import sklearn.svm
import seaborn as sns
import scipy.io as sio
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings(action = 'ignore')
```

[ ]:
```
mat = sio.loadmat('./data/data2.mat')
print(mat.keys())

data = pd.DataFrame(mat.get('X'), columns=['X1', 'X2'])
data['y'] = mat.get('y')
data.head()
```

```
dict_keys(['__header__', '__version__', '__globals__', 'X', 'y'])
```

[ ]:

	X1	X2	y
0	0.107143	0.603070	1
1	0.093318	0.649854	1
2	0.097926	0.705409	1
3	0.155530	0.784357	1
4	0.210829	0.866228	1

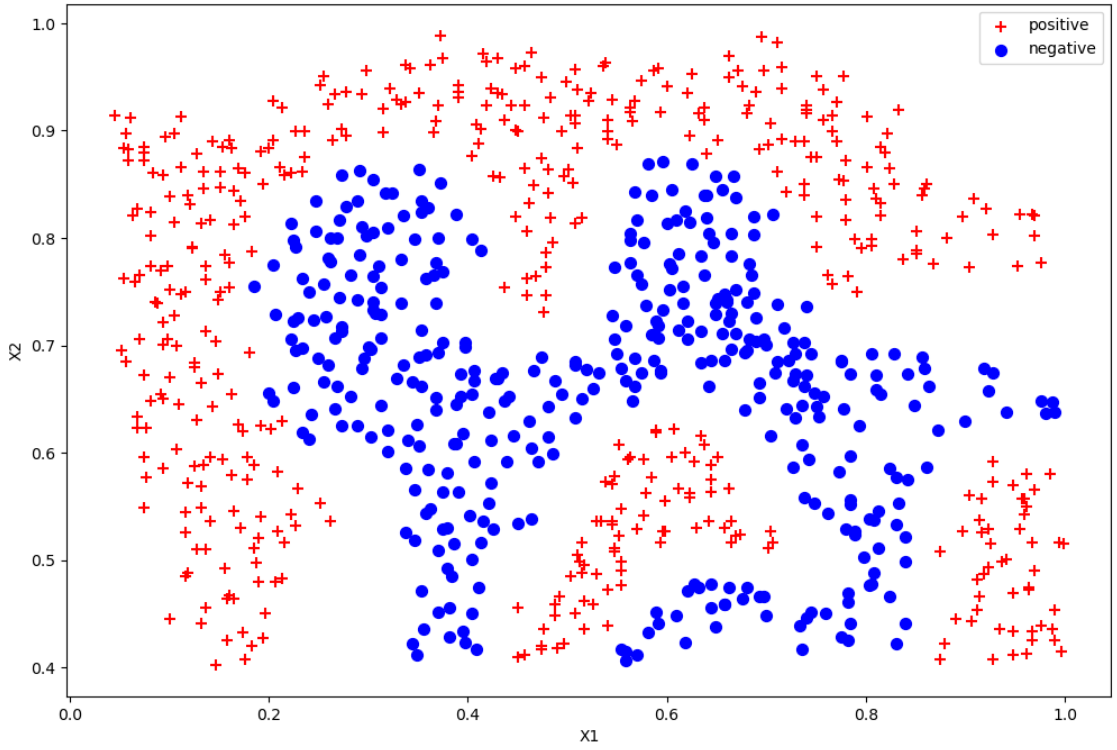
[7]:
```
data.shape
min(data['X1']), max(data['X1']), min(data['X2']), max(data['X2'])
```

[7]: (0.0449309, 0.998848, 0.402632, 0.988596)

Visualize data

[8]:
```
positive = data[data.y == 1]
negative = data[data['y'] == 0]

fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(positive['X1'], positive['X2'], label='positive', s=50, marker='+',␣
 ↪c='r')
ax.scatter(negative['X1'], negative['X2'], label='negative', s=50, marker='o',␣
 ↪c='b')
ax.legend(loc='best')
ax.set_xlabel('X1')
ax.set_ylabel('X2')
plt.show()
```



2.1 Classify the all data into two classes with Radial basis function (RBF) kernel and compute the accuracy of the classifier. [6 points]

- **Use `svm.SVC()` with parameter `c=100` and set the `gamma` parameter to 1 and 10, respectively.**

[9]:
```
########## START YOUR CODE HERE #############
svc1 = sklearn.svm.SVC(C=100, kernel='rbf', gamma=1, probability=True)
svc1.fit(data[['X1', 'X2']], data['y'])
print(svc1.score(data[['X1', 'X2']], data['y']))
svc10 = sklearn.svm.SVC(C=100, kernel='rbf', gamma=10, probability=True)
svc10.fit(data[['X1', 'X2']], data['y'])
print(svc10.score(data[['X1', 'X2']], data['y']))
########## END YOUR CODE HERE ##############
```

```
0.8342989571263036
0.9698725376593279
```

2.2 Visualize the decision boundary for the above two cases. [6 points]

- **You are likely to use `plt.contour()`**

Case 1: gamma=1

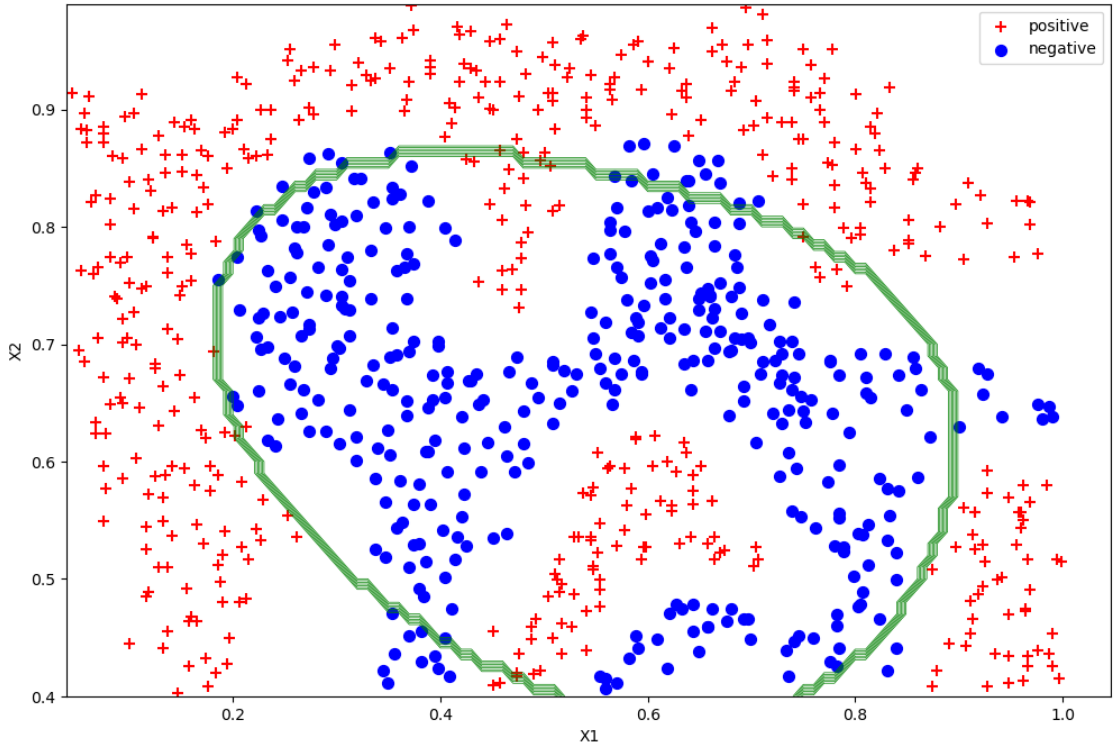
[10]:
```
positive = data[data.y == 1]
negative = data[data['y'] == 0]

fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(positive['X1'], positive['X2'], label='positive', s=50, marker='+',␣
 ↪c='r')
ax.scatter(negative['X1'], negative['X2'], label='negative', s=50, marker='o',␣
 ↪c='b')
ax.legend(loc='best')
ax.set_xlabel('X1')
ax.set_ylabel('X2')


x1 = np.arange(0.04, 1, 0.01)
x2 = np.arange(0.4, 1, 0.01)
x1, x2 = np.meshgrid(x1, x2)


########## START YOUR CODE HERE #############
y_pred1 = np.array([svc1.predict(np.vstack((a, b)).T) for (a, b) in zip(x1,␣
 ↪x2)])
plt.contour(x1, x2, y_pred1, colors='g', linewidths=.5)
```

```
########## END YOUR CODE HERE ###############
plt.show()
```



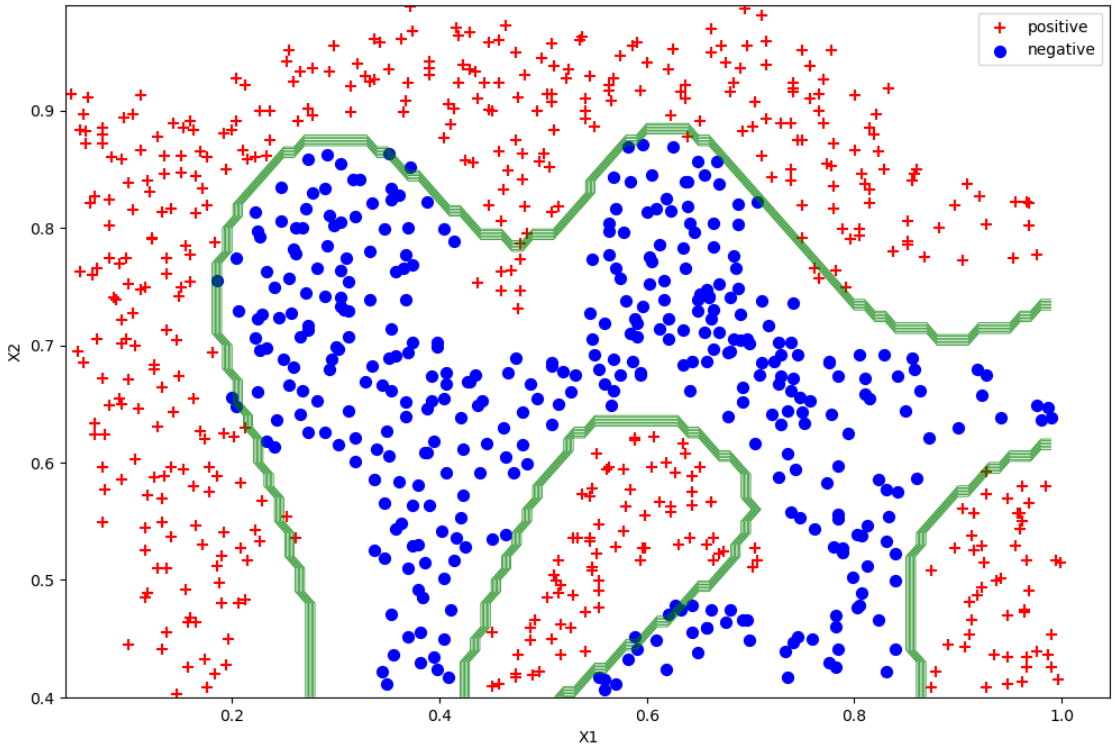
Case 2: gamma=10

[11]:
```
positive = data[data.y == 1]
negative = data[data['y'] == 0]

fig, ax = plt.subplots(figsize=(12, 8))
ax.scatter(positive['X1'], positive['X2'], label='positive', s=50, marker='+',␣
 ↪c='r')
ax.scatter(negative['X1'], negative['X2'], label='negative', s=50, marker='o',␣
 ↪c='b')
ax.legend(loc='best')
ax.set_xlabel('X1')
ax.set_ylabel('X2')


x1 = np.arange(0.04, 1, 0.01)
x2 = np.arange(0.4, 1, 0.01)
x1, x2 = np.meshgrid(x1, x2)
```

```
########## START YOUR CODE HERE #############
y_pred10 = np.array([svc10.predict(np.vstack((a, b)).T) for (a, b) in zip(x1,␣
 ↪x2)])
plt.contour(x1, x2, y_pred10, colors='g', linewidths=.5)
########## END YOUR CODE HERE ###############
plt.show()
```



5. [20 points] [*Principal Component Analysis (PCA)*] As discussed in class, Principal Component Analysis (PCA) can be derived from a reconstruction error perspective. Consider the following formulation:

Given a dataset $\{\mathbf{x}^t\}_{t=1}^N$ where $\mathbf{x}^t \in \mathbb{R}^d$, we aim to represent each data point in a lower-dimensional space and then reconstruct it with minimal error. The reconstruction model is:

$$\hat{\mathbf{x}}^t = \mathbf{m} + \mathbf{W}\mathbf{z}^t$$

where $\mathbf{m} \in \mathbb{R}^d$ is a common bias vector, $\mathbf{W} \in \mathbb{R}^{d\times k}$ is the projection matrix, and $\mathbf{z}^t \in \mathbb{R}^k$ is the low-dimensional representation of $\mathbf{x}^t$. The optimization problem is to minimize the total reconstruction error:

$$\text{(P-RE)} \quad \underset{\mathbf{m},\mathbf{W},\{\mathbf{z}^t\}}{\text{minimize}} \quad \frac{1}{2}\sum_{t=1}^{N} \left\|\mathbf{x}^t - (\mathbf{m} + \mathbf{W}\mathbf{z}^t)\right\|_2^2$$
$$\text{subject to} \quad \mathbf{W}^T\mathbf{W} = \mathbf{I}_k$$

Assume we center the data by setting $\mathbf{m}$ to be the sample mean, i.e., $\mathbf{m} = \frac{1}{N}\sum_{t=1}^N \mathbf{x}^t$, and define the centered data points as $\tilde{\mathbf{x}}^t = \mathbf{x}^t - \mathbf{m}$.

(a) [4 points] Show that for a fixed $\mathbf{W}$, the optimal low-dimensional representation $\mathbf{z}^t$ for each data point is given by the projection $\mathbf{z}^t = \mathbf{W}^T\tilde{\mathbf{x}}^t$.

(b) [8 points] Substitute this optimal $\mathbf{z}^t$ back into the objective function of (P-RE). Show that the problem of finding the optimal $\mathbf{W}$ reduces to:

$$\underset{\mathbf{W}}{\text{maximize}} \quad \sum_{t=1}^{N} \tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t \quad \text{subject to} \quad \mathbf{W}^T\mathbf{W} = \mathbf{I}_k$$

(c) [8 points] Further show that this maximization problem is equivalent to finding the k eigenvectors of the sample covariance matrix $\mathbf{\Sigma} = \frac{1}{N}\sum_{t=1}^N \tilde{\mathbf{x}}^t\tilde{\mathbf{x}}^{tT}$ corresponding to its k largest eigenvalues. (**Hint:** Use the property of the cyclic property of the trace $\text{tr}(\mathbf{AB}) = \text{tr}(\mathbf{BA})$.)

Solution:

(a) With $\mathbf{m}$ fixed as the sample mean, the centered data is $\tilde{\mathbf{x}}^t = \mathbf{x}^t - \mathbf{m}$. The reconstruction error for a single point becomes:

$$\left\|\mathbf{x}^t - (\mathbf{m} + \mathbf{W}\mathbf{z}^t)\right\|_2^2 = \left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{z}^t\right\|_2^2$$

For a fixed $\mathbf{W}$, we minimize this error with respect to $\mathbf{z}^t$. This is a least squares problem. Expanding the norm:

$$\left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{z}^t\right\|_2^2 = (\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{z}^t)^T(\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{z}^t) = \tilde{\mathbf{x}}^{tT}\tilde{\mathbf{x}}^t - 2\tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{z}^t + \mathbf{z}^{tT}\mathbf{W}^T\mathbf{W}\mathbf{z}^t$$

Using the constraint $\mathbf{W}^T\mathbf{W} = \mathbf{I}$, the last term simplifies to $\mathbf{z}^{tT}\mathbf{z}^t$. Taking the gradient with respect to $\mathbf{z}^t$ and setting it to zero:

$$\nabla_{\mathbf{z}^t}\left[-2\tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{z}^t + \mathbf{z}^{tT}\mathbf{z}^t\right] = -2\mathbf{W}^T\tilde{\mathbf{x}}^t + 2\mathbf{z}^t = \mathbf{0}$$

Solving for $\mathbf{z}^t$ gives the optimal representation:

$$\mathbf{z}^t = \mathbf{W}^T\tilde{\mathbf{x}}^t \quad \square$$

(b) Substitute the optimal $\mathbf{z}^t = \mathbf{W}^T\tilde{\mathbf{x}}^t$ back into the reconstruction error for a single point:

$$\left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{z}^t\right\|_2^2 = \left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t\right\|_2^2$$

The total objective function becomes:

$$\frac{1}{2}\sum_{t=1}^{N}\left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t\right\|_2^2$$

Minimizing this is equivalent to maximizing its negative (ignoring the constant factor 1/2). Let's expand the squared norm:

$$\left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t\right\|_2^2 = \tilde{\mathbf{x}}^{tT}\tilde{\mathbf{x}}^t - 2\tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t + \tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t$$

Using the constraint $\mathbf{W}^T\mathbf{W} = \mathbf{I}$ twice, the last term simplifies to $\tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t$. Thus:

$$\left\|\tilde{\mathbf{x}}^t - \mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t\right\|_2^2 = \tilde{\mathbf{x}}^{tT}\tilde{\mathbf{x}}^t - \tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t$$

Therefore, minimizing the sum of reconstruction errors is equivalent to:

$$\underset{\mathbf{W}}{\text{maximize}} \quad \sum_{t=1}^{N} \tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t \quad \text{subject to} \quad \mathbf{W}^T\mathbf{W} = \mathbf{I} \quad \square$$

(c) Let $\mathbf{\Sigma} = \frac{1}{N}\sum_{t=1}^{N} \tilde{\mathbf{x}}^t\tilde{\mathbf{x}}^{tT}$ be the sample covariance matrix. The objective function can be rewritten using the trace operator:

$$\sum_{t=1}^{N} \tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t = \sum_{t=1}^{N} \text{tr}\left(\tilde{\mathbf{x}}^{tT}\mathbf{W}\mathbf{W}^T\tilde{\mathbf{x}}^t\right) = \sum_{t=1}^{N} \text{tr}\left(\mathbf{W}^T\tilde{\mathbf{x}}^t\tilde{\mathbf{x}}^{tT}\mathbf{W}\right)$$

By the linearity of trace, this equals:

$$\text{tr}\left(\mathbf{W}^T\left(\sum_{t=1}^{N} \tilde{\mathbf{x}}^t\tilde{\mathbf{x}}^{tT}\right)\mathbf{W}\right) = N \cdot \text{tr}\left(\mathbf{W}^T\mathbf{\Sigma}\mathbf{W}\right)$$

Let $\mathbf{W} = [\mathbf{w}_1, \ldots, \mathbf{w}_k]$, where each $\mathbf{w}_i$ is a column vector. Then:

$$\text{tr}\left(\mathbf{W}^T\mathbf{\Sigma}\mathbf{W}\right) = \sum_{i=1}^{k} \mathbf{w}_i^T\mathbf{\Sigma}\mathbf{w}_i$$

The constraint $\mathbf{W}^T\mathbf{W} = \mathbf{I}$ implies that the columns of $\mathbf{W}$ are orthonormal. The classical result in PCA is that this sum is maximized when $\{\mathbf{w}_i\}_{i=1}^k$ are the eigenvectors of $\mathbf{\Sigma}$ corresponding to the k largest eigenvalues.